

실시간 위성지도 + 오늘의 별자리 지도 웹사이트 상세 명세서

1. 프로젝트 개요

1.1 프로젝트명

StarEarth (가칭)

1.2 프로젝트 목적

사용자가 웹 브라우저에서 다음을 직관적으로 볼 수 있는 시각화 웹사이트를 만든다. 1. 구 형태의 3D 지구 2. 실시간 또는 준실시간 인공위성의 위치와 궤도 3. 오늘 날짜 기준의 별자리 지도 4. 감성적이고 몰입감 있는 시각 효과를 통해 “천문/지구/위성” 정보를 아름답게 표현

1.3 최종 사용자 경험 목표

- 지구를 회전하며 위성과 별자리를 함께 감상할 수 있다.
- 현재 시간 또는 선택한 날짜에 맞춰 위성 위치와 별자리 정보를 확인할 수 있다.
- 단순 정보 표시가 아니라, “보는 재미”가 강한 인터랙티브한 화면을 제공한다.

1.4 프로젝트 성격

- 단순 CRUD 웹사이트가 아니라 **3D 시각화 중심 웹 애플리케이션**
- 백엔드는 데이터 제공과 계산, 프론트엔드는 시각화에 집중
- 초반에는 정확도보다 **구현 완성도와 화면 완성도**를 우선

2. 기술 선택 명세

2.1 백엔드 프레임워크

권장: FastAPI

선택 이유

- API 서버를 만들기에 구조가 단순하고 빠르다.
- 비동기 처리와 실시간 데이터 제공에 유리하다.
- 프론트엔드와 백엔드가 분리되는 구조에 잘 맞는다.
- 위성 데이터 갱신, 천문 계산 API, 날짜 기반 조회 API를 만들기 좋다.

Django를 선택해도 되는 경우

- 관리자 페이지가 매우 중요하다.
- DB 기반의 전통적인 서비스 구조가 중심이다.
- 인증, 권한, 사용자 관리, 게시판 같은 기능이 먼저 필요하다.

이 프로젝트에서의 결론

- 백엔드 중심 작업은 FastAPI가 더 적합
- Django는 익숙하지만, 이 프로젝트는 “화면 + API + 계산” 성격이 강해서 FastAPI 학습 가치가 높다.

2.2 프론트엔드

- Three.js를 핵심 렌더링 엔진으로 사용
- 필요시 React를 붙여서 컴포넌트 기반 UI 구성
- 프론트엔드는 아래 역할을 맡는다.
- 3D 지구 렌더링
- 카메라 제어
- 위성 표시
- 별자리 표시
- 날짜 선택 UI
- 애니메이션과 인터랙션

2.3 3D/천문 관련 라이브러리 후보

- Three.js: 3D 렌더링
- Astropy: 천문 계산
- Skyfield: 위성/천문 계산 보조
- SGP4: TLE 기반 위성 궤도 계산
- Numpy: 수치 연산
- Pydantic: API 데이터 모델링

2.4 데이터 저장 방식

초기 버전은 DB를 꼭 강제하지 않고 시작한다.

- 초기: 외부 API + 서버 캐시 + 메모리 처리
- 확장: PostgreSQL 또는 SQLite

권장 순서

1. MVP 단계: SQLite 또는 메모리 캐시
2. 확장 단계: PostgreSQL

3. 전체 기능 범위 정의

3.1 핵심 기능

1. 3D 지구 렌더링
2. 위성 위치 표시
3. 위성 궤도 선 표시
4. 오늘 날짜의 별자리 지도 표시
5. 날짜 변경에 따른 별자리/하늘 변화
6. 마우스 회전, 확대/축소, 자동 회전

3.2 확장 기능

1. 위성 클릭 시 이름, 고도, 속도, 궤도 정보 표시
2. 별자리 클릭 시 이름과 주요 별 정보 표시
3. 낮/밤 테마 자동 전환
4. 도시/위치 기준 하늘 변경
5. 실시간 갱신 주기 조정
6. 즐겨찾기 위성/별자리 저장
7. 2D 미니맵 또는 HUD 추가

3.3 제외 범위(초기 버전)

- 로그인/회원가입
 - 결제
 - 커뮤니티
 - 위성 예측의 고급 정밀 시뮬레이션
 - 실제 천문관 수준의 정밀 성도 전체 구현
-

4. 화면 구성 명세

4.1 메인 화면 구조

화면은 크게 4개 영역으로 나눈다.

A. 3D 뷰포트 영역

- 화면의 중심
- 구형 지구가 렌더링됨
- 위성은 지구 주위를 움직이는 점 또는 아이콘으로 표시
- 별자리는 지구 바깥 또는 배경 하늘에 오버레이 형태로 표시

B. 상단 정보 바

- 현재 날짜/시간 표시
- 현재 모드 표시: 지구 / 위성 / 별자리 / 전체
- 새로고침 상태 또는 데이터 업데이트 시각 표시

C. 좌측 제어 패널

- 날짜 선택
- 시간 선택
- 위성 표시 토글
- 별자리 표시 토글
- 궤도 선 표시 토글
- 자동 회전 토글
- 배경 밝기/스타일 선택

D. 우측 상세 패널

- 선택된 위성 정보
- 선택된 별자리 정보

- 현재 위치 설명
- 관련 수치

4.2 화면 상태

- 기본 상태: 아무것도 선택되지 않음
- 위성 선택 상태: 정보 패널이 위성 중심으로 바뀜
- 별자리 선택 상태: 별자리 설명이 강조됨
- 날짜 변경 상태: 재계산 중 로딩 표시

4.3 반응형 정책

- 데스크톱: 3D 뷰 + 좌우 패널 표시
- 태블릿: 좌측 패널 축소, 우측 패널 접힘
- 모바일: 3D 뷰 우선, 패널은 바텀시트 형태

5. 사용자 흐름 명세

5.1 최초 진입 흐름

1. 사용자가 사이트에 접속한다.
2. 로고와 함께 지구가 서서히 나타난다.
3. 배경 별이 점차 등장한다.
4. 위성 데이터와 별자리 데이터가 로딩된다.
5. 기본 날짜는 오늘로 설정된다.
6. 사용자는 지구를 회전하며 위성과 별자리를 본다.

5.2 날짜 변경 흐름

1. 사용자가 날짜 선택기를 연다.
2. 날짜를 선택한다.
3. 프론트엔드가 백엔드 API에 요청한다.
4. 백엔드가 해당 날짜 기준 별자리/천문 데이터를 계산한다.
5. 화면의 별자리와 관련 표시가 갱신된다.

5.3 위성 클릭 흐름

1. 사용자가 지구 주위의 위성을 클릭한다.
2. 위성의 이름, 높이, 속도, 궤도 정보가 표시된다.
3. 해당 위성의 궤도 선이 강조된다.

5.4 별자리 클릭 흐름

1. 사용자가 별자리 영역을 클릭한다.
2. 별자리 이름과 주요 별 데이터가 표시된다.
3. 해당 별자리 선과 별이 강조된다.

6. 아키텍처 명세

6.1 권장 전체 구조

```
Frontend (Three.js / React)
    ↓ HTTP API
Backend (FastAPI)
    ↓
천문 계산 / 위성 궤도 계산 / 데이터 정리
    ↓
외부 데이터 소스 (TLE, 천문 데이터)
```

6.2 역할 분리

프론트엔드

- 화면 렌더링
- 애니메이션
- 사용자 입력 처리
- API 호출
- 시각적 강조

백엔드

- 위성 좌표 계산용 데이터 제공
- 날짜별 별자리 데이터 제공
- 정적/동적 리소스 API 제공
- 계산 로직 집중

6.3 개발 원칙

- 화면과 계산을 분리한다.
- UI는 프론트에서 처리하고, 계산은 백엔드에서 처리한다.
- 초기에는 정확한 데이터보다 “움직이고 보이는 것”을 먼저 완성한다.

7. 데이터 명세

7.1 위성 데이터 항목

위성 하나당 아래 정보를 가진다.

- id
- name
- catalog_number
- latitude
- longitude
- altitude_km
- velocity_km_s

- inclination
- epoch_time
- orbit_type
- tle_line1
- tle_line2
- updated_at

7.2 별자리 데이터 항목

별자리 하나당 아래 정보를 가진다.

- id
- name
- abbreviation
- season
- visible_months
- major_stars
- star_positions
- line_segments
- description
- mythology_summary

7.3 날짜/시간 기준 데이터 항목

- target_date
- target_time
- timezone
- observer_location(선택)
- sky_rotation_angle
- visible_constellations

8. API 명세

8.1 공통 규칙

- 응답 형식은 JSON
- 상태 코드는 명확히 구분
- 실패 시 에러 메시지를 표준화

8.2 위성 목록 API

GET /api/satellites

현재 표시할 위성 목록을 반환한다.

응답 예시

```
{
  "success": true,
```

```
"data": [
  {
    "id": 1,
    "name": "ISS",
    "latitude": 12.34,
    "longitude": 56.78,
    "altitude_km": 408,
    "velocity_km_s": 7.66
  }
]
```

8.3 특정 위성 상세 API

GET /api/satellites/{satellite_id}

- 위성의 상세 정보 반환
- 궤도 정보와 TLE 포함 가능

8.4 위성 궤도 API

GET /api/satellites/{satellite_id}/orbit

- 궤도 선을 표시하기 위한 좌표 배열 반환

8.5 별자리 목록 API

GET /api/constellations?date=YYYY-MM-DD

- 해당 날짜 기준으로 표시할 별자리 목록 반환

8.6 별자리 상세 API

GET /api/constellations/{constellation_id}

- 별자리 이름, 선분, 주요 별, 설명 반환

8.7 천문 상태 API

GET /api/sky?date=YYYY-MM-DD&time=HH:MM:SS

- 날짜/시간 기준 하늘 상태 정보 반환
- 별자리 회전 값, 기본 천구 상태, 시각 효과용 값 제공

8.8 헬스체크 API

GET /api/health

- 서버 정상 동작 확인

9. 프론트엔드 렌더링 명세

9.1 지구 렌더링

- 완전한 구형 Mesh 사용
- 지구 텍스처 적용
- 구름 레이어 또는 광원 효과 추가 가능
- 밤면 라이트 효과 선택 적용 가능

9.2 위성 렌더링

- 작은 점, 아이콘, 혹은 발광 점으로 표시
- 위성 궤도는 라인으로 표시 가능
- 위성이 지나간 경로에 잔상 효과를 넣을 수 있음

9.3 별자리 렌더링

- 별은 점으로 렌더링
- 별자리는 점과 선 연결로 표시
- 중요 별은 크기를 키움
- 별자리 이름은 라벨로 표시

9.4 카메라 동작

- 마우스 드래그: 회전
- 휠: 확대/축소
- 더블클릭: 해당 객체 포커스
- 자동 회전: 옵션으로 토글

9.5 시각 효과

- 발광(Glow)
- 부드러운 전환 애니메이션
- 배경 별점 무작위 분포
- 선택 객체 하이라이트

10. 백엔드 계산 명세

10.1 위성 좌표 계산

입력

- TLE 데이터
- 기준 시각

출력

- 위성의 지구 중심 좌표 또는 위도/경도/고도

처리

- SGP4 기반 계산 적용
- 특정 시각에 맞는 위치 산출
- 화면 렌더링이 가능한 3D 좌표로 변환

10.2 별자리 계산

입력

- 기준 날짜
- 기준 시각
- 관측 위치(선택)

출력

- 해당 날짜에 맞는 별자리 각도, 배치, 강조값

처리

- 천구 좌표계 또는 별자리 맵 좌표 계산
- 날짜에 따라 약간의 회전 또는 노출 차이를 반영
- 실제 천문 정확도가 부족한 초기 버전에서는 고정된 대표 별자리 데이터로 시작 가능

10.3 좌표 변환

- 위도/경도/고도 → 3D 좌표
- 천구 좌표 → 3D 배경 좌표
- 지구 반지름 기준 상대 좌표 계산

11. 데이터 소스 명세

11.1 위성 데이터

- TLE 형식 사용
- 초기에는 대표 위성 몇 개만 사용
- 이후 확장 시 여러 위성군 지원

11.2 별자리 데이터

- 초기에는 12개 황도12궁 또는 대표 별자리로 시작 가능
- 이후 88개 공식 별자리로 확장 가능

11.3 데이터 갱신 정책

- 위성 데이터: 일정 주기마다 갱신
- 별자리 데이터: 기본적으로 고정, 날짜/시간에 따른 계산만 수행

12. 개발 단계별 구현 명세

12.1 1단계: 프로젝트 뼈대 만들기

목표

- 실행 가능한 기본 웹앱 생성
- FastAPI 서버 동작
- 프론트엔드 페이지 연결

해야 할 일

- 프로젝트 폴더 구조 생성
- FastAPI 기본 라우트 작성
- 정적 파일/템플릿 연결
- 첫 화면 렌더링

완료 기준

- 브라우저에서 접속 시 빈 페이지 또는 환영 화면 표시
-

12.2 2단계: 3D 지구 구현

목표

- 구형 지구 렌더링 성공

해야 할 일

- Three.js 세팅
- 구체 생성
- 지구 텍스처 적용
- 조명 추가
- 카메라와 마우스 컨트롤 추가

완료 기준

- 화면에서 지구가 부드럽게 회전하고 확대/축소 가능
-

12.3 3단계: 배경 우주 구현

목표

- 우주 느낌의 배경 구성

해야 할 일

- 별 배경 추가
- 미세한 점광원 배치
- 배경 그라데이션 또는 노이즈 효과 추가

완료 기준

- 지구만 떠 있는 느낌이 아니라 우주 공간처럼 보임
-

12.4 4단계: 위성 1개 표시

목표

- 최소 1개 위성을 지구 주변에 표시

해야 할 일

- 위성 데이터 하드코딩 또는 mock API 사용
- 위성 3D 위치 변환
- 위성 마커 렌더링
- 선택 시 정보 표시

완료 기준

- ISS 같은 위성 하나가 지구 주변에 떠 있음
-

12.5 5단계: 위성 궤도선 표시

목표

- 위성의 이동 경로를 시각화

해야 할 일

- 궤도 좌표 샘플링
- 라인 객체 렌더링
- 위성 이동 애니메이션

완료 기준

- 위성 점과 궤도 선이 함께 보임
-

12.6 6단계: 날짜별 데이터 연결

목표

- 날짜를 바꾸면 데이터가 갱신됨

해야 할 일

- 날짜 선택 UI 구현
- 백엔드 날짜 파라미터 처리
- 재계산된 데이터 화면 반영

완료 기준

- 오늘 날짜와 다른 날짜를 선택했을 때 시각이 바뀜
-

12.7 7단계: 별자리 기본 표시

목표

- 별자리 점과 선 표시 성공

해야 할 일

- 대표 별자리 데이터 준비
- 별 위치 렌더링
- 별자리 연결선 표시
- 라벨 표시

완료 기준

- 별자리 하나 이상이 예쁘게 보임
-

12.8 8단계: 오늘 날짜 별자리 지도

목표

- 오늘 날짜 기준으로 별자리 정보가 보여짐

해야 할 일

- 현재 날짜 자동 반영
- 날짜 기준 별자리 목록 반환 API 연결
- 시각적으로 강조된 지도처럼 표현

완료 기준

- 사이트에 들어가면 오늘 날짜에 맞는 별자리 정보가 자동으로 표시됨

12.9 9단계: 인터랙션 강화

목표

- 사용자가 객체를 직접 조작 가능

해야 할 일

- 위성 클릭 이벤트
- 별자리 클릭 이벤트
- 포커싱 애니메이션
- 툴팁 또는 사이드 패널 개선

완료 기준

- 클릭했을 때 반응이 확실하게 나타남
-

12.10 10단계: 디자인 고도화

목표

- “엄청 이쁘게” 보이도록 완성도 향상

해야 할 일

- Glow 효과
- 트레일 효과
- 부드러운 모션
- 폰트와 색감 통일
- 어두운 배경 기반의 우주 테마 디자인

완료 기준

- 기능뿐 아니라 감성적인 비주얼이 살아남
-

12.11 11단계: 성능 최적화

목표

- 렌더링과 API 응답이 안정적으로 동작

해야 할 일

- API 캐싱
- 렌더링 객체 수 조절
- 불필요한 재렌더 최소화

- 데이터 로딩 시 skeleton 또는 로딩 스피너 적용

완료 기준

- 화면이 버벅이지 않고 부드럽게 동작
-

12.12 12단계: 배포

목표

- 인터넷에서 접근 가능한 서비스로 공개

해야 할 일

- 백엔드 배포
- 프론트엔드 빌드 및 배포
- 환경변수 설정
- 도메인 연결(선택)

완료 기준

- 외부에서 접속 가능한 최종 사이트 완성
-

13. 폴더 구조 명세

13.1 백엔드 구조 예시

```
backend/  
  app/  
    main.py  
  api/  
  core/  
  services/  
  models/  
  schemas/  
  utils/  
  data/
```

13.2 프론트엔드 구조 예시

```
frontend/  
  src/  
    components/  
    pages/  
    hooks/
```

```
utils/  
assets/  
styles/
```

13.3 역할 요약

- api: 라우터
- services: 계산과 비즈니스 로직
- models: DB 모델
- schemas: 요청/응답 타입
- utils: 공통 함수

14. 품질 기준

14.1 기능 품질

- 각 기능이 독립적으로 동작해야 한다.
- 날짜 변경, 위성 갱신, 별자리 표시가 서로 충돌하지 않아야 한다.

14.2 시각 품질

- 색감이 통일되어야 한다.
- 화면이 복잡해 보여도 정보 우선순위가 명확해야 한다.
- 중요한 정보는 과하지 않게 강조해야 한다.

14.3 유지보수 품질

- 위성 데이터와 별자리 데이터를 분리해서 관리한다.
- 계산 로직과 렌더링 로직을 섞지 않는다.
- 나중에 기능을 추가하기 쉬운 구조로 만든다.

15. MVP 정의

15.1 반드시 포함될 것

- FastAPI 서버
- 3D 지구
- 위성 1개 이상
- 별자리 1개 이상
- 날짜 선택 기능
- 기본 UI 패널

15.2 MVP 성공 기준

- 사용자가 접속했을 때 지구와 위성, 별자리를 동시에 볼 수 있다.
- 날짜를 바꾸면 화면이 갱신된다.
- 최소한의 인터랙션이 가능하다.

16. 향후 확장 로드맵

16.1 단기 확장

- 위성 수 증가
- 별자리 수 증가
- 툴팁 개선
- 로딩 애니메이션 정교화

16.2 중기 확장

- 위치 기반 하늘 표현
- 시간대별 별자리 변화
- 사용자 즐겨찾기
- 검색 기능

16.3 장기 확장

- 실제 천문 교육용 서비스
- AR/VR 연동
- 모바일 최적화 강화
- 외부 천문 데이터 통합

17. 추천 개발 순서

1. FastAPI 프로젝트 생성
2. 프론트엔드 기본 화면 구성
3. Three.js로 지구 렌더링
4. 카메라 제어 추가
5. 위성 1개를 점으로 표시
6. 위성 궤도선 추가
7. 별자리 점과 선 표시
8. 날짜 선택 UI 연결
9. 백엔드 계산 API 연결
10. 디자인과 효과 고도화
11. 성능 최적화
12. 배포

18. 최종 결론

이 프로젝트는 **백엔드보다 프론트엔드 3D 시각화 비중이 큰 프로젝트**다. 따라서 처음부터 완벽한 천문 정확도를 추구하기보다,

- 먼저 화면이 아름답게 돌아가고
- 위성이 보이고
- 별자리가 보이고
- 날짜에 따라 변화하는 구조를 만든 뒤 정확도를 높이는 방향이 가장 현실적이다.

즉, 개발 순서는 다음 한 줄로 요약된다.

“FastAPI로 데이터를 받고, Three.js로 우주를 그리며, 천문 계산은 점점 정교하게 만든다.”